

ESCOLA POLITÉCNICA DA UNIVERSIDADE DE SÃO PAULO
Departamento de Engenharia de Computação e Sistemas Digitais

PCS3732 – Laboratório de Processadores



Projeto Final - Scheduler de memória virtual

Celso Tadaki Sinoka	13682851
Felipe Luís Körbes	13682893
Davi Félix de Lima	13680230

Sumário

Introdução.....	3
1. Memória virtual.....	3
2. Ajustes no Linker Script e Estrutura de Código.....	4
3. Decifrando o Coprocessador 15.....	7
3.1. O Coprocessador 15 e o SCTLR: (System Control Register):.....	7
3.2. Funcionamento do Switch.....	9
3.3. MMU_STOP.....	10
3.4. TLB Invalida.....	10
3.5. MMU Start.....	11
3.5.1. DACR - as permissões de domínio de memória.....	11
3.5.2. Lidando com o mapeamento da TTB:.....	12
3.5.3. Ligando a MMU.....	12
4. Procedimento.....	13
4.1. Fluxo de execução.....	13
4.2. Uso de alocação dinâmica com stub _sbrk.....	16
4.3. Funções de gerenciamento da memória virtual.....	17
4.3.1 Função map_section.....	17
4.3.2 Função map_invalid.....	18
4.3.3 Função mmu_flat.....	18
5. Problemas encontrados e soluções.....	19
6. Metodologia de testes e isolamento da memória.....	19
7. Programa final de exemplo.....	20
7.1. Ilustrações da execução.....	21
7.1.1. Processo 1.....	22
7.1.2. Processo 2.....	23
7.1.3. Processo 3.....	25
8. Limitações e sugestões de incrementos.....	26
8.1. Limitações.....	26
8.2. Melhorias.....	27
9. Conclusão.....	28

Introdução

Este relatório descreve o funcionamento de um escalonador básico de processos para suportar memória virtual em um sistema ARM (Raspberry Pi com núcleos Cortex-A7). O objetivo do projeto foi permitir que múltiplos processos executassem com espaços de endereçamento isolados, usando a Unidade de Gerência de Memória (MMU) do ARM. Em resumo, adotou-se um esquema simples de paginação por seções de 1 MB, onde a seção de código de cada processo (.text) é mapeada de forma direta (mesmo endereço físico e virtual), enquanto as regiões de dados e pilha de cada processo são mapeadas dinamicamente em diferentes seções físicas de 1 MB reservadas a cada processo. Assim, cada processo enxerga um espaço de endereços virtuais próprios, como se tivesse memória dedicada.

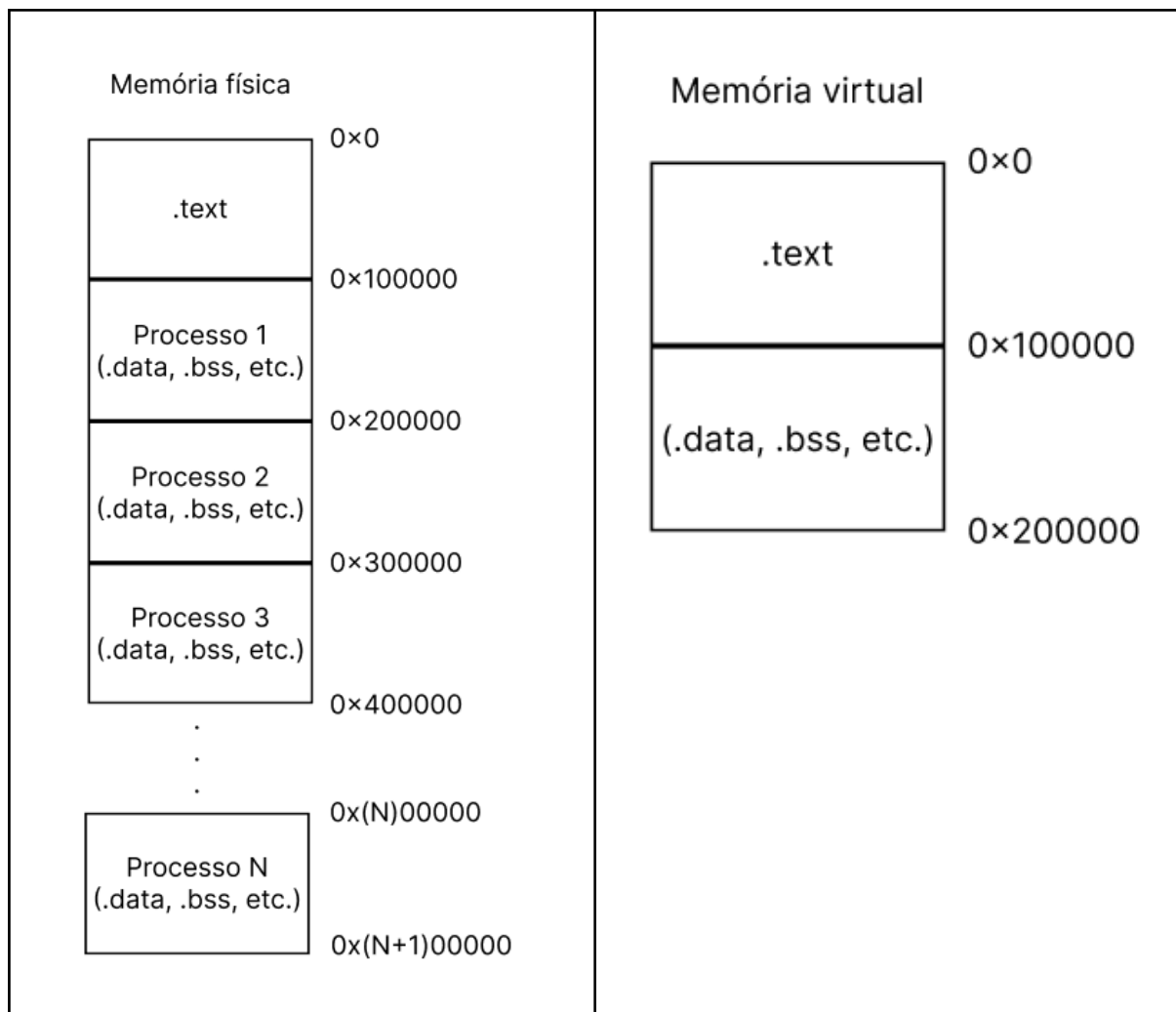
1. Memória virtual

No projeto, foi implementado um esquema de memória virtual para isolar os espaços de endereçamento de múltiplos processos em um sistema embarcado ARM. A estratégia central foi a paginação por seções de 1 MB, utilizando a Unidade de Gerenciamento de Memória (MMU) do processador.

A principal característica deste sistema é que a seção de código de cada processo (.text) é compartilhada e mapeada de forma direta, ou seja, o endereço físico é o mesmo que o virtual. Em contrapartida, as áreas de dados (.data, .bss) e a pilha (stack) de cada processo são mapeadas dinamicamente em diferentes seções físicas de 1 MB, que são reservadas individualmente para cada processo.

Dessa forma, embora cada processo opere com a ilusão de possuir seu próprio espaço de endereçamento virtual dedicado, na realidade, a MMU está traduzindo esses endereços virtuais (VA, *virtual addresses*) para locais distintos na memória física (PA, *physical addresses*). Por exemplo, o mesmo endereço virtual para uma variável global em processos diferentes apontará para endereços físicos distintos, garantindo o isolamento e prevenindo que um processo interfira nos dados de outro.

A imagem a seguir ilustra a organização da memória física e a perspectiva da memória virtual de cada processo:



Como demonstrado, na memória física, os dados de cada processo ocupam uma seção de 1 MB sequencial e exclusiva. No entanto, do ponto de vista de qualquer processo, sua área de dados e heap/pilha sempre parece estar localizada a partir do endereço virtual.

2. Ajustes no Linker Script e Estrutura de Código

Para viabilizar a implementação do scheduler com memória virtual, foi necessário modificar o linker script e reorganizar algumas seções de memória no código fonte.

O linker script foi alterado e definiu-se o `.text` logo após a região de boot e, em seguida, fez-se o linker pular para o endereço `0x100000` (1 MB) antes de definir as seções de `.rodata`, `.data` e `.bss`. Dessa forma, todos os dados globais da aplicação

ficam a partir do endereço físico 0x100000, que corresponde à primeira seção de dados de um processo. No arquivo de script de ligação, isso aparece como:

```
SECTIONS {
    .boot 0x8000: {
        *(.boot)
    }

    . = ALIGN(8);
    .text : {
        *(.text)
    }

    . = 0x100000;

    . = ALIGN(8);
    .rodata : { *(.rodata) }

    . = ALIGN(8);
    data_begin = .;
    .data : { *(.data) }
    data_end = .;

    . = ALIGN(8);
    bss_begin = .;
    .bss : { *(.bss*) }
    bss_end = .;

    .heapstack : {
        . = ALIGN(8);
        heap_addr = .;
        . = . + 32K;
        . = . + 4K;
    }
}
```

```

    stack_user = .;
}

. = 0x200000;

. = 0x500000;
.tcb : { *(.tcb) }
. = . + 16K;
stack_sys = .;
. = . + 4K;
stack_irq = .;
. = 0x600000;
}

```

Uma outra alteração que pôde ser feita no *linker script* é a remoção dos diferentes stack pointers para cada um dos processos. Antes, como a memória não era virtualizada, caso não houvesse um stack pointer cada cada processo, um processo iria interferir na pilha do outro e os dados seriam corrompidos. Agora, esse endereço pode ser o mesmo (inclusive, deve ser o mesmo), pois processos diferentes referenciando esse mesmo endereço irão mapeá-lo para diferentes áreas da memória, não havendo conflito.

Para evitar conflitos ao remapear a janela de dados/pilha dos processos (VA 0x100000–0x1FFFFFF), as pilhas do modo SVC e IRQ foram movidas para um endereço não contido dentro do espaço de endereçamento virtual dos processos; aqui, é necessário tomar cuidado para não mapear nenhum processo para essa seção da pilha SVC, que deve ser mantida intacta para o correto funcionamento do programa.

Na seção `.heapstack`, estão definidos os símbolos para os endereços iniciais do *heap* e da *stack* do modo usuário, sendo o espaço entre eles usado para armazenar o conteúdo de cada uma dessas seções de memória: a pilha (*stack*) cresce para endereços menores enquanto o *heap* cresce para endereços maiores.

Além disso, foi criada uma região específica do scheduler: a seção .tcb é fixada em duas seções de 1MB da memória física a partir de 0x500000, acrescentam-se 16 KB. Nela, são posicionadas todas as variáveis relacionadas ao scheduler e também a pilha do modo supervisor, ou seja, todas as variáveis que são utilizadas no programa enquanto a memória virtual está desligada e, portanto, não devem estar no espaço de endereçamento virtual 0x000000-0x1FFFFFF.

3. Decifrando o Coprocessador 15

Para podermos entender melhor o que acontece no código que estamos utilizando como base do projeto. Precisamos olhar para os blocos principais e analisar passo a passo o que está acontecendo. Podemos afirmar que isso será importante para dar confiança ao que estamos depurando e sobre o comportamento esperado.

Estamos principalmente olhando para o Coprocessador 15 e o seu funcionamento, especificamente para o `Raspberry Pi BCM2836`. E para este caso, temos núcleos ARM Cortex-A7. Essas são informações importantes para podermos checar referências.

3.1. O Coprocessador 15 e o SCTL: (System Control Register):

[Neste link](#) você consegue mais detalhes sobre o registrador em questão. Todo o nosso projeto envolve lidar com o MMU, e sua TLB, para definir como será o mapeamento da memória, para isso, será utilizado o registrador chamado SCTL do coprocessador C15. Estes elementos serão manipulados nas instruções seguintes.

Bits	Name	Function
[31]	-	Reserved.
[30]	TE	Thumb Exception enable.
[29]	AFE	Access flag enable bit.
[28]	TRE	TEX remap enable bit.
[27:26]	-	Reserved.
[25]	EE	Exception Endianness bit.
[24:21]	-	Reserved.
[20]	UWXN	Unprivileged write permission implies PL1 Execute Never (XN).
[19]	WXN	Write permission implies Execute Never (XN).
[18:14]	-	Reserved.
[13]	V	Vectors bit.
[12]	I	Instruction cache enable bit.
[11]	Z	Branch prediction enable bit.
[10]	SW	SWP and SWPB enable bit.
[9:3]	-	Reserved.
[2]	C	Cache enable bit.
[1]	A	Alignment bit.
[0]	M	Address translation enable bit.

Temos registradores no coprocessador p15. O registrador c1 tem vários bits de controle importantes: O bit 0 é o que controla o MMU. O bit 2 controla a cache dos dados, e o bit 12 controla a cache das instruções. Estes são os pontos mais importantes que veremos aqui.

Mas, se você quiser ir mais a fundo no entendimento, mostro abaixo como funciona, de maneira geral as instruções dos coprocessadores, pode-se ver mais [aqui](#).

Encoding T1

Armv7-M

MCR<C> <coproc>, <opc1>, <Rt>, <CRn>, <CRm>{, <opc2>}

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	1	1	0	opc1	0	CRn					Rt	coproc					opc2	1	CRm								

```
t = UInt(Rt); cp = UInt(coproc);  
if t == 15 || t == 13 then UNPREDICTABLE;
```

Encoding T2

Armv7-M

MCR2<C> <coproc>, <opc1>, <Rt>, <CRn>, <CRm>{, <opc2>}

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	1	1	0	opc1	0	CRn					Rt	coproc					opc2	1	CRm								

```
t = UInt(Rt); cp = UInt(coproc);  
if t == 15 || t == 13 then UNPREDICTABLE;
```

Assembler syntax

MCR{2}<C><Q> <coproc>, #<opc1>, <Rt>, <CRn>, <CRm>{, #<opc2>}

where:

- 2 If specified, selects the C == 1 form of the encoding. If omitted, selects the C == 0 form.
- <C><Q> See [Standard assembler syntax fields on page A7-177](#).
- <coproc> Specifies the name of the coprocessor. The standard generic coprocessor names are p0-p15.
- <opc1> Is a coprocessor-specific opcode in the range 0-7.
- <Rt> Is the Arm register whose value is transferred to the coprocessor.
- <CRn> Is the destination coprocessor register.
- <CRm> Is an additional destination coprocessor register.
- <opc2> Is a coprocessor-specific opcode in the range 0-7. If it is omitted, <opc2> is assumed to be 0.

3.2. Funcionamento do Switch

A troca entre processos é disparada por software com yield() (gera swi #0 com código de operação 1). No manipulador swi em switch.s, as interrupções são desabilitadas, o contexto do processo atual é salvo no TCB (save_context) e a função C trata_swi(op) é chamada. Para op=1, ela chama schedule(). O caminho por interrupção (irq) também salva o contexto, incrementa ticks e invoca schedule(). Esse processo será mais detalhado na seção 4.

Em sched.c, schedule() primeiro chama mmu_stop(), avança o ponteiro da lista (head = head->next), atualiza tid e tcb para o processo selecionado e remapeia a janela virtual 0x100000–0x1FFFFFF para a seção física do processo atual com:

```
map_section((ttb_l1_t*)0x3EFFC000, 0x100000, 0x100000 * (tid + 1), 0x0);
```

3.3. MMU_STOP

A função MMU_STOP tem a função de desabilitar o MMU e os caches. A primeira linha: ("mrc p15, 0, r2, c1, c0, 0") serve para ler o registrador SCTLR e colocar o seu valor no registrador r2. Particularmente, o registrador em questão p15 tem vários bits de controle importantes, que você pode ver no link fornecido.

Todos estes códigos, inclusive o MMU_STOP, leem o SCTLR com mrc, modificam os bits de controle e depois escrevem de volta com mcr. A instrução bic ('bit clear') é usada para zerar bits específicos fazendo um AND com o complemento da máscara: em ARMv7, bic Rd, Rn, Operand2 computa $Rd = Rn \& \sim \text{Operand2}$. No nosso caso, aplicamos máscaras imediatas para limpar os bits 12, 2 e 0. Isso é equivalente a $r2 = r2 \& \sim(0x1000 \mid 0x0004 \mid 0x0001)$. Em seguida, mcr grava o novo valor no SCTLR.

Para finalizar, este novo valor é passado para o registrador SCTLR, efetivamente alterando o estado de controle do sistema do coprocessador 15. Para esclarecer, $\#0x1000$ é o valor 4096 em decimal que coincide ao bit 1 shiftado de 12: $1 \ll 12 = 4096$. A mesma lógica segue para as demais linhas.

```
void mmu_stop(void) {
    asm volatile(
        _("mrc p15, 0, r2, c1, c0, 0")
        _("bic r2, #0x1000")
        _("bic r2, #0x0004")
        _("bic r2, #0x0001")
        _("mcr p15, 0, r2, c1, c0, 0")
    );
}
```

O bit 0, 2 e 12 são desligados - desligando a função de MMU, Data Cache e Instruction Cache.

3.4. TLB Invalida

Realiza a invalidação da TLB (translation lookaside buffer) que é o cache para a tradução de endereços lógicos para físicos. E aqui é onde atua a barreira de

memória, que serve para a sincronização dos modos de funcionamento. Fato que será indispensável de entender para o projeto. Temos referências do uso idêntico desta linha 2 para lidar com o desligamento do TLB. [Aqui no Stack Exchange](#) temos um código com um propósito parecido com o de nosso projeto que fala um pouco sobre isso, embora com pouco sucesso.

```
start_mmu:
mov r2, #0
mcr p15, 0, r2, c8, c7, 0 ;@ invalidate tlb
```

Este trecho invalida uma TLB que é unificada, porque ela traduz tanto o endereço de instruções quanto de dados. Modificar c8 e c7 com o opcode 0, e com o valor 0 (do r2) é o que invalida todas as entradas do TLB.

Logo abaixo, é feito `mcr p15, 0, r2, c7, c10, 4`. Este caso especificamente é para lidar com o cache de dados, o DSB (Data Synchronization Barrier), que é uma barreira específica de sincronização. Esta linha serve para fazer com que o CPU espere por todas as operações com dados, que estão na memória, sejam concluídas antes de prosseguir. - É basicamente um flush na pipe e assegura a ordem correta dos dados (estamos cansados de falar disso em OArC).

```
void tlb_invalida(void) {
asm volatile(
_("mov r2, #0")
_("mcr p15, 0, r2, c8, c7, 0")
_("mcr p15, 0, r2, c7, c10, 4")
::: "r2");
}
```

Invaldar a TLB é importante, porque se ela estivesse válida, a CPU iria procurar por endereços que agora estão inválidos. Isso significa que toda a vez que desligarmos a MMU, e o mapeamento de memória virtual, precisamos dar este flush, para que haja a sincronização adequada e evitarmos confusões!

3.5. MMU Start

3.5.1. DACR - as permissões de domínio de memória.

O `mvn` é uma instrução do ARM que faz um *move* de um valor negado (move not), então no registrador `r2`, depois da primeira linha, será carregada com `0xFFFFFFFF`. Esta, e a próxima linha estão adequando a trilha de bits para alterar na linha 2.

Pesquisando um pouco mais, vemos que o registrador `c3` do coprocessador 15, na linha 3 se trata do Domain Access Control Register. E ele faz justamente a definição de quais são as permissões para cada um dos 16 domínios de memória. Basicamente ele altera o DACR definindo que o domínio 0 (das páginas de memória do kernel) fica com o código 11 - para permissão irrestrita ao domínio de memória em questão.

- 00 - no access at all
- 01 - access checked against values in TLB entry (user)
- 11 - access always permitted (admin)

Para mais informações sobre DACR, acesse [esse link](#), e também achamos esse [post no Stack Overflow](#) que pode ser interessante.

3.5.2. Lidando com o mapeamento da TTB:

Na quarta e quinta linhas do código assembly deste bloco, faremos a manipulação do registrador `C2` do coprocessador CP15. E vemos que este registrador se trata da TTBR (Translation Table Base Registers). Registers em plural, porque nesta arquitetura temos dois, indicadas por `c2, c0, 0` e `c2, c0, 1`. `%0` é o argumento `*ttb` que está sendo passado na função, é um ponteiro que aponta para a tabela de mapeamento de primeiro nível.

Trocando em miúdos, estes comandos vão colocar o endereço da tabela de tradução de páginas definida em memória nos Translation Table Base Registers (TTBRs) da máquina. Assim, efetivamente os endereços base da tabela de páginas passam a ficar coerentes. O ARMv7 tem dois registradores do tipo TTBR porque um é voltado para o espaço de usuário e o outro é para os espaços referenciados ao kernel.

3.5.3. Ligando a MMU

Por fim, as seguintes 4 linhas vão servir para ler o registrador SCTLR de que já falamos anteriormente no ponto sobre MMU Stop. Primeiro lemos este valor para `r2`,

modificamos alguns dos seus valores e aí retornarmos para o registrador. É de se imaginar que tudo isso vai fazer o exato inverso do mmu stop, e vai mesmo. Setar os bits 0, 2 e 12 como ligados, ligando MMU, e os caches de instrução e de dados.

Dado que manipulamos os registradores de TTB no item [2.5.2](#), agora o endereçamento está finalmente modificado adequadamente - ou seja, manipulamos a TTB com uma nova base, e isso sim, efetivamente (com a MMU ligada para tradução) é o que liga a Memória Virtual.

```
void mmu_start(ttb_l1_t *ttb) {
    asm volatile(
        _("mvn r2, #0")
        _("bic r2, #0xC")
        _("mcr p15, 0, r2, c3, c0, 0")
        _("mcr p15, 0, %0, c2, c0, 0")
        _("mcr p15, 0, %0, c2, c0, 1")
        _("mrc p15, 0, r2, c1, c0, 0")
        _("orr r2, r2, #0x05")
        _("orr r2, r2, #0x1000")
        _("mcr p15, 0, r2, c1, c0, 0")
        :: "r"(ttb) : "r2");
}
```

Ao final deste bloco, o sistema fica com a memória virtual habilitada. Parece simples, mas é interessante entender o que está por trás de tudo isso.

4. Procedimento

4.1. Fluxo de execução

O programa começa na função *system_main*, ainda no modo SVC, fazendo um mapeamento *flat* de toda a memória física, ainda com o a memória virtual desligada. Para isso, basta preencher a tabela de páginas mapeando cada seção da memória virtual para exatamente a mesma seção da memória física, sendo o efeito prático desse mapeamento a impressão de que a memória virtual está desligada mesmo quando ela estiver habilitada, já que VA=PA no mapeamento de endereços.

Também pode ser feita a inicialização mapeando de maneira *flat* apenas as seções utilizadas e mapeando as seções não utilizadas como inválidas.

O próximo passo é inicializar as estruturas de dados do scheduler. Para isso, na função *sched_init* são criadas 3 TCBs, uma para cada processo, sendo que o PC (r15) é inicializado com o valor do símbolo definido pela função do processo (user1_main, user2_main ou user3_main), o SP (r13) é inicializado com o valor do label para a pilha definido no *linker script* e o CPSR é inicializado com todas as flags zeradas e o modo definido como usuário (0b10000). A estrutura de dados da TCB pode ser vista abaixo, sendo composta por cada um dos 17 registradores do processo (r1-15 e cpsr) e também o endereço do heap naquele processo, necessário para suportar alocação dinâmica. O endereço do *heap* é salvo como *char* pois ele deve ser representado por 1 byte.

```
typedef struct tcb_s {
    uint32_t regs[17]; // Contexto (r0-r15, cpsr)
    char *heap;
} tcb_t;
```

Estrutura de dados da TCB de cada processo.

Essas TCBs são então inseridas em uma lista ligada seguindo a estrutura mostrada abaixo, que contém a TCB referente ao respectivo nó, o endereço para o próximo nó e o TID do processo.

```
typedef struct tcb_ll {
    tcb_t tcb;
    int tid;
    volatile struct tcb_ll *next;
} tcb_ll_t;
```

Estrutura de dados da lista ligada de TCBs.

Tanto o nó quanto a TCB atuais são armazenados em variáveis globais no programa, que são respectivamente chamadas de *head* e *tcb*, mostradas abaixo. Depois de inserir todas as TCBs na lista, a função *sched_init* também inicia essas variáveis com os valores correspondentes ao primeiro processo que será executado.

```
/*
 * Macro para definir uma seção específica para uma função
 */
#define SECTION(X) __attribute__((section(X)))
```

```

/**
 * Lista estática dos tasks definidos no sistema.
 */

SECTION(".tcb") volatile tcb_ll_t tcb_list[MAX_TASKS];
SECTION(".tcb") volatile tcb_ll_t *head = &tcb_list[0];

/*
 * Variáveis globais, acessadas em boot.s
 */
SECTION(".tcb") volatile int tid;
SECTION(".tcb") volatile tcb_t *tcb;

```

Depois de configurar o scheduler, o programa chama a subrotina *task_switch*. Uma boa prática aqui caso não fosse feito o *mmu_flat* é mapear a memória virtual para o primeiro processo escalonado com a função *map_section*. Ela é responsável por recuperar o contexto a partir da variável global *tcb* e preencher os registradores com os valores que lá estavam salvos. É importante perceber que o PC (r15) do processo sendo recuperado deve inicialmente ser posicionado no LR (r14) para que o programa não pule abruptamente para o processo na troca de contexto. No fim, a instrução `movs pc, lr` move a execução do programa para o processo recuperado da TCB, sendo o sufixo S importante para poder preencher o SPSR com o CPSR, o que permite que o processador volte para o modo usuário devido ao fato de o CPSR das TCBS terem sido configurados para o modo usuário.

Em seguida, a memória virtual é ligada e o processo começa a executar, já referenciando endereços que serão remapeados pelo MMU para as seções da memória física configuradas na tabela de páginas.

Como citado anteriormente na seção 3, para um processo ceder sua vez a outro, é usada a função *yield()*, que é responsável por fazer a chamada de uma *software interrupt* (*swi*) com parâmetro r0=1 para troca de contexto. No tratamento dessa interrupção, o modo de execução muda para SVC e a primeira operação a ser realizada é o salvamento do processo atual na sua respectiva TCB, pois ele está prestes a ser desescalonado pela chamada da função *schedule*, responsável por escalonar o próximo processo.

Na função *schedule*, a memória virtual é desligada e o endereço que o programa referencia passa a ser diretamente o PA (aqui valor serão referenciados os valores da seção *.tcb*). Em seguida são atualizadas as variáveis *tcb* e *head* e *tid* para os dados do próximo processo e o mapeamento da seção do processo atual é realizado, sendo mapeada a seção 0x100000 da memória para sua correspondente seção na memória virtual. Para isso, ele obtém o TID do processo atual pela variável global *head* e então calcula a seção física correspondente. Quando a função *schedule* retorna, a função de tratamento de interrupção segue para a subrotina *task_switch*, que recupera o contexto a partir da TCB atualizada, inicializa a memória virtual e continua para o próximo processo escalonado.

4.2. Uso de alocação dinâmica com stub *_sbrk*

Um ajuste importante que precisou ser feito para utilização de alocação dinâmica com *malloc* é a mudança da implementação trivial do stub *_sbreak* ("system break" ou "segment break"). Esse *stub* é utilizado pela libc para atualizar o valor atual do heap na alocação dinâmica de memória dentro da seção *.heapstack*.

A implementação trivial desse *stub* utiliza um endereço de heap armazenado em uma variável global que muda a cada chamada do *stub*; a cada vez que mais espaço do heap é modificado, é adicionado ou subtraído nessa variável o tamanho que deve ser alocado ou desalocado, respectivamente.

No entanto, para o scheduler com memória virtual isso não funciona, uma vez que cada processo terá o seu heap e, caso seja usado uma mesma variável global para o endereço do heap, quando um processo atualiza o heap depois do outro, o segundo processo irá ser atualizado a partir do valor do heap deixado pelo primeiro, gerando um espaço de memória vazio que não será alocado no endereçamento do segundo processo.

Para isso, o valor do endereço do heap para cada processo foi recuperado em sua respectiva TCB e recuperado na implementação do *_sbrk*. Lembrando que a variável *heap* deve ser representada por *char* para que o incremento do endereço representado por ela seja feito em bytes.

```
void *_sbrk(int incr) {  
    unsigned char *heap = tcb->heap;
```



```

    unsigned char *prev_heap;

    prev_heap = heap;

    heap += incr;
    return prev_heap;
}

```

Nova implementação do `_sbrk`

4.3. Funções de gerenciamento da memória virtual

Para realizar o mapeamento da memória virtual, foram utilizadas as função no código da aula 9, cuja documentação já foi feita na seção 3. Portanto, foi necessário transformar essas funções em uma biblioteca para utilizá-las. Ou seja, foi necessário criar uma pasta *libmem* com os arquivos *mem.c* e *mem.h* e, em seguida, realizar a compilação com os comandos abaixo para obter o arquivo *libmem.a*. Para que essa utilização como biblioteca pudesse ser feita, a única alteração necessária no código foi o recebimento do endereço da tabela de páginas como parâmetro nas funções que a utilizam.

```

arm-none-eabi-gcc -o mem.o -march=armv7-a -mtune=cortex-a7 -g
-lgcc -c -L/usr/lib/gcc/arm-none-eabi/13.2.1/ mem.c

arm-none-eabi-ld -M=libmem.map mem.o -lgcc
-L/usr/lib/gcc/arm-none-eabi/13.2.1/ -i -o libmem.a

```

O arquivo *libmem.a* é então referenciado no arquivo principal com os parâmetros `-lmem -L./libmem`. As funções *mmu_start()*, *mmu_stop()* e *tlb_invalida()* já foram explicadas, agora bastando explicar a função *map_section()* e *map_flat()*.

4.3.1 Função *map_section*

A função mais importante é a *map_section*, que segue abaixo:

```

void map_section(ttb_l1_t* ttb_l1, uint32_t virt, uint32_t fis,
uint32_t flags) {
    int indice = (virt & 0xffff0000) >> 20;
    int descr = (fis & 0xffff0000) | 0xc002 | flags;
}

```

```
ttb_l1->descriptor[indice] = descr;
}
```

A estrutura de dados da tabela de páginas é mostrada abaixo, sendo composta basicamente por um conjunto de 4096 registradores, ou seja, um descritor para cada uma das 4K ($4 \cdot 2^{10}$) seções de 1MB (2^{20}) que a memória possui.

```
typedef struct {
    uint32_t descriptor[4096];
} ttb_l1_t;
```

A função *map_section* tem a função de mapear a seção representada pelo endereço *virt* para a seção da memória física representada pelo endereço *fis*. Para isso, é necessário o índice da tabela de páginas correspondente à seção *virt* com: (i) o endereço da da seção a qual aquele endereço virtual será mapeado na memória física, (ii) as flags relacionadas àquele endereço, sendo o descritor composto por (i) e (ii).

Para obter o índice da seção das memórias física e virtual, basta obter os 12 bits mais significativos de cada endereço, com os quais é possível referenciar todas as 4G (2^{12}) seções da memória. Os bits menos significativos servem como *offset*, ou seja, um valor que pode ser somado ao endereço base da seção.

Com os índices calculadores, basta salvar o valor do índice de memória física e as respectivas flags do mapeamento no descritor da tabela de páginas referenciado pelo índice da memória virtual (nos 12 bits mais significativos) concatenados com as flags do mapeamento.

4.3.2 Função *map_invalid*

A função *map_invalid* apenas salva o valor 0 no descritor referenciado pelo índice do endereço virtual, invalidando esse endereço virtual utilizando a função *map_section*.

4.3.3 Função *mmu_flat*

A função *map_section* faz um *loop* por todas as 4Giga seções da memória e mapeia a seção virtual *i* para a mesma seção física *i* utilizando a função *map_section*

5. Problemas encontrados e soluções

Durante a integração da MMU ao escalonador, três pontos críticos apareceram e foram resolvidos em sequência. O primeiro foi um travamento intermitente ao retornar do `schedule()`. Ao remapear a janela virtual de dados/pilha (VA 0x100000–0x1FFFFFF) para o bloco físico do processo corrente, o kernel acabava operando sobre uma pilha que podia cair nessa mesma região remapeada. A decisão foi tirar a pilha do modo SVC da área sujeita a troca de mapeamento. No linker script, além da `.heapstack`, criamos a região do scheduler: a seção `.tcb` é fixada em 0x500000 e, +16 KB adiante, define-se `stack_sys`, reservando uma pilha de sistema fora da janela remapeada. No caminho de retorno em ASM (`task_switch`), o contexto é restaurado e a paginação é religada, assegurando que a volta ao usuário ocorra com a memória virtual no estado esperado. O travamento deixou de ocorrer.

O segundo problema foi a corrupção de metadados de escalonamento. Variáveis como `tcb`, `head`, `tid` e a própria lista de TCBs estavam no espaço de endereçamento virtual, que era remapeado para outra seção da memória física na qual essas variáveis não estavam presentes, o que explicava inconsistências ao restaurar o contexto. A correção foi mover as estruturas do escalonador para a seção dedicada `.tcb`, posicionada pelo linker em 0x500000 e, portanto, fora da janela de endereçamento virtual 0x100000–0x1FFFFFF. Com isso, os descritores permaneceram estáveis independentemente do processo ativo.

6. Metodologia de testes e isolamento da memória

Durante a integração da MMU ao escalonador, foi validado o isolamento entre processos e a estabilidade das trocas de contexto em três frentes. Primeiro, usamos um teste didático com variáveis globais: em `main.c` há globais (`a`, `b`) acessadas por três tarefas (`user1_main`, `user2_main`, `user3_main`). Cada tarefa grava um valor distinto em `a` e executa laços curtos com `yield()`. Como o projeto mantém a seção `.text` fixa no primeiro 1 MB da memória física e remapeia apenas a faixa de endereçamento 0x100000–0x1FFFFFF para a seção física do processo corrente, o mesmo VA usado para a variável `a` aponta para PAs diferentes por processo. O

efeito esperado, e observado nesse arranjo, é que cada tarefa enxerga o valor que gravou, sem interferência entre elas.

Estruturas do escalonador e metadados foram movidos para uma seção fora do espaço de endereçamento virtual, como já mencionado, portanto, não são mapeados no espaço dos processos e não sofrem remapeamento nas trocas. Na prática, isso impede que uma tarefa escreva nas TCBs ao alternar de contexto e evita que a pilha do SVC/"kernel" caia em área remapeada.

7. Programa final de exemplo

Para ilustrar o funcionamento, definimos 3 processos de teste para serem executados e escalonados. Cada um desses processos, representados pelas funções `user1_main`, `user2_main` e `user3_main`, irá utilizar duas variáveis globais definidas no programa principal, chamadas `a` e `b`. Como essas variáveis foram definidas e não inicializadas no código, elas irão para a seção `.bss` e, como explicado anteriormente, vão portanto cair na seção `0x100000-0x1FFFFFF` da memória. Assim, elas serão mapeadas pela tabela de memória virtual e serão particulares para cada processo.

A variável `a` é um array de inteiros, enquanto a variável `b` é um ponteiro para inteiro, que será utilizado como um array dinamicamente alocado. Assim, podemos validar o funcionamento da memória virtual, inclusive com uso de alocação dinâmica.

Cada um dos processos irá:

- Chamar uma subrotina para somar dois inteiros e armazenando resultado em `C`
- armazenar 1024 inteiros de mesmo valor entre si no array definido por `a`;
- armazenar uma determinada quantidade de inteiros de mesmo valor entre si a partir do endereço apontado por `b`.

O valor dos inteiros armazenados em `a`, o valor e quantidade de elementos armazenados em `b` e a soma realizada variará entre cada um dos processos:

	user1_main	user2_main	user3_main
Inteiro armazenado em a	0xAAAAAAAA	0BBBBBBBBB	0CCCCCCCCC
Inteiro armazenado em b	0xA1A1A1A1	0xB2B2B2B2	0xC3C3C3C3
Quantidade de elementos armazenados em b	5	10	100
Soma realizada	1+1	1+2	1+3

Assim, poderemos ver que, em cada processo, as variáveis **a** e **b** terão valores diferentes mesmo estando sendo referenciadas pelo mesmo endereço.

A troca de processos é feita de modo não preemptivo: os processos concedem controle ao outro processo no começo da função de soma no fim de uma interação do loop infinito dentro deles. A ideia de desescalar o processo no meio da função de soma é ver que a pilha sendo utilizada no endereço definido por *stack_user* no *linker script* continuará funcionando separadamente para cada um dos processos.

A ferramenta utilizada para isso foi o GDB por meio de breakpoints e do comando **x**, que permite inspecionar valores de memória. A fim de facilitar a depuração, adicionamos os breakpoints diretamente no makefile a partir de uma variável que é referenciada por todos os comandos que rodam o GDB.

7.1. Ilustrações da execução

Abaixo, seguem algumas imagens ao longo da execução dos processos no GDB para cada um dos processos.

7.1.1. Processo 1

Tabela de páginas no processo user1_main (comando x/4096wx 0x3EFFC000), equivalente a mapeamento flat:

0x3effc000:	0x0000c002	0x0010c002	0x0020c002	0x0030c002
0x3effc010:	0x0040c002	0x0050c002	0x0060c002	0x0070c002
0x3effc020:	0x0080c002	0x0090c002	0x00a0c002	0x00b0c002
0x3effc030:	0x00c0c002	0x00d0c002	0x00e0c002	0x00f0c002
0x3effc040:	0x0100c002	0x0110c002	0x0120c002	0x0130c002
0x3effc050:	0x0140c002	0x0150c002	0x0160c002	0x0170c002
0x3effc060:	0x0180c002	0x0190c002	0x01a0c002	0x01b0c002
0x3effc070:	0x01c0c002	0x01d0c002	0x01e0c002	0x01f0c002
0x3effc080:	0x0200c002	0x0210c002	0x0220c002	0x0230c002
0x3effc090:	0x0240c002	0x0250c002	0x0260c002	0x0270c002

Pilha da soma durante o processo 1, mostrando os parâmetro 1 e 1 (comando x/3wx \$sp):

0x10a118:	0x00000001	0x00000001	0x0010a13c
-----------	------------	------------	------------

Variáveis *a*, *b* e *c*, respectivamente após execução do loop interno do processo user1_main, após a execução da soma no processo user3_main o

x/1024wx a:

0x10012c <a>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x10013c <a+16>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x10014c <a+32>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x10015c <a+48>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x10016c <a+64>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x10017c <a+80>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x10018c <a+96>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x10019c <a+112>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x1001ac <a+128>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x1001bc <a+144>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa
0x1001cc <a+160>:	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa	0xaaaaaaaa

x/100wx b:

```
0x101148: 0xa1a1a1a1 0xa1a1a1a1 0xa1a1a1a1 0xa1a1a1a1
0x101158: 0xa1a1a1a1 0x00000000 0x00000000 0x00000000
0x101168: 0x00000000 0x00000000 0x00000000 0x00000000
0x101178: 0x00000000 0x00000000 0x00000000 0x00000000
0x101188: 0x00000000 0x00000000 0x00000000 0x00000000
0x101198: 0x00000000 0x00000000 0x00000000 0x00000000
0x1011a8: 0x00000000 0x00000000 0x00000000 0x00000000
0x1011b8: 0x00000000 0x00000000 0x00000000 0x00000000
0x1011c8: 0x00000000 0x00000000 0x00000000 0x00000000
0x1011d8: 0x00000000 0x00000000 0x00000000 0x00000000
0x1011e8: 0x00000000 0x00000000 0x00000000 0x00000000
```

p c:

```
(gdb) p c
$4 = 2
```

Percebe-se que a memória mantém os valores esperados, como se a estivesse isolado dos outros processos.

7.1.2. Processo 2

Tabela de páginas no processo no processo user2_main (agora os 3 primeiros nibbles do descritor no índice 1 valem 0x002, indicando que houve mapeamento):

```
0x3effc000: 0x000c002 0x0020c002 0x0020c002 0x0030c002
0x3effc010: 0x0040c002 0x0050c002 0x0060c002 0x0070c002
0x3effc020: 0x0080c002 0x0090c002 0x00a0c002 0x00b0c002
0x3effc030: 0x00c0c002 0x00d0c002 0x00e0c002 0x00f0c002
0x3effc040: 0x0100c002 0x0110c002 0x0120c002 0x0130c002
0x3effc050: 0x0140c002 0x0150c002 0x0160c002 0x0170c002
0x3effc060: 0x0180c002 0x0190c002 0x01a0c002 0x01b0c002
0x3effc070: 0x01c0c002 0x01d0c002 0x01e0c002 0x01f0c002
0x3effc080: 0x0200c002 0x0210c002 0x0220c002 0x0230c002
0x3effc090: 0x0240c002 0x0250c002 0x0260c002 0x0270c002
0x3effc0a0: 0x0280c002 0x0290c002 0x02a0c002 0x02b0c002
```

Pilha da soma no processo user2_main:

```
(gdb) p $rsp
0x10a118: 0x00000002 0x00000001 0x0010a13c
```

Variáveis *a*, *b* e *c*, respectivamente após execução do loop interno do processo user2_main, após a execução do loop interno do processo user1_main:

x/1024wx a

0x10012c <a>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x10013c <a+16>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x10014c <a+32>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x10015c <a+48>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x10016c <a+64>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x10017c <a+80>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x10018c <a+96>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x10019c <a+112>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x1001ac <a+128>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x1001bc <a+144>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb
0x1001cc <a+160>:	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb	0xbbbbbbbb

x/100wx b

0x101148:	0xb2b2b2b2	0xb2b2b2b2	0xb2b2b2b2	0xb2b2b2b2
0x101158:	0xb2b2b2b2	0xb2b2b2b2	0xb2b2b2b2	0xb2b2b2b2
0x101168:	0xb2b2b2b2	0xb2b2b2b2	0x00000000	0x00000000
0x101178:	0x00000000	0x00000000	0x00000000	0x00000000
0x101188:	0x00000000	0x00000000	0x00000000	0x00000000
0x101198:	0x00000000	0x00000000	0x00000000	0x00000000
0x1011a8:	0x00000000	0x00000000	0x00000000	0x00000000
0x1011b8:	0x00000000	0x00000000	0x00000000	0x00000000
0x1011c8:	0x00000000	0x00000000	0x00000000	0x00000000
0x1011d8:	0x00000000	0x00000000	0x00000000	0x00000000
0x1011e8:	0x00000000	0x00000000	0x00000000	0x00000000

p c

\$5 = 3

7.1.3. Processo 3

Tabela de páginas no processo no processo user3_main (agora os 3 primeiros nibbles do descritor no índice 1 valem 0x003, indicando que houve mapeamento):

0x3effc000:	0x0000c002	0x0030c002	0x0020c002	0x0030c002
0x3effc010:	0x0040c002	0x0050c002	0x0060c002	0x0070c002
0x3effc020:	0x0080c002	0x0090c002	0x00a0c002	0x00b0c002
0x3effc030:	0x00c0c002	0x00d0c002	0x00e0c002	0x00f0c002
0x3effc040:	0x0100c002	0x0110c002	0x0120c002	0x0130c002
0x3effc050:	0x0140c002	0x0150c002	0x0160c002	0x0170c002
0x3effc060:	0x0180c002	0x0190c002	0x01a0c002	0x01b0c002
0x3effc070:	0x01c0c002	0x01d0c002	0x01e0c002	0x01f0c002
0x3effc080:	0x0200c002	0x0210c002	0x0220c002	0x0230c002
0x3effc090:	0x0240c002	0x0250c002	0x0260c002	0x0270c002
0x3effc0a0:	0x0280c002	0x0290c002	0x02a0c002	0x02b0c002

Pilha da soma no processo user2_main:

0x10a118:	0x00000003	0x00000001	0x0010a13c
-----------	------------	------------	------------

Variáveis *a*, *b* e *c*, respectivamente após execução do loop interno do processo user2_main, após a execução do loop interno do processo user1_main:

x/1024wx a

0x10012c <a>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x10013c <a+16>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x10014c <a+32>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x10015c <a+48>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x10016c <a+64>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x10017c <a+80>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x10018c <a+96>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x10019c <a+112>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x1001ac <a+128>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x1001bc <a+144>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff
0x1001cc <a+160>:	0xffffffff	0xffffffff	0xffffffff	0xffffffff

x/100wx b

```
0x101148: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x101158: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x101168: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x101178: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x101188: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x101198: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x1011a8: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x1011b8: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x1011c8: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x1011d8: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
0x1011e8: 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3 0xc3c3c3c3
--Type <RET> for more, q to quit, c to continue without paging--
```

p c

```
$6 = 4
```

8. Limitações e sugestões de incrementos

8.1. Limitações

- Devido a problemas para testar o programa com a interrupção de timer para troca de contexto no GDB, optamos por deixar os processos executando de maneira não preemptiva, ou seja, eles devem explicitamente liberar a CPU para a execução de outros programas.
- Como a seção `.text` não é remapeada, o código de todos os processos rodando, o gerenciamento deles (scheduling) e toda a gestão de inicialização e mapeamento de memória devem caber em apenas 1MB, o que limita o tamanhos dos processos que podem rodar no projeto.
- O mapeamento de dados é estático, ou seja, o processo com TID $i=0,1,2,\dots$ usa a seção física $(i+1) * 0x100000$ (0x100000, 0x200000, 0x300000, ...), não havendo bitmap/lista de seções livres da memória física para que haja uma alocação dinâmica de espaços da memória física.
- A proteção do kernel é feita, essencialmente, por não mapear as áreas do escalonador (ex.: `.tcb` em 0x500000 e `stack_sys`) no espaço dos processos. Não há configuração de AP bits/domínios para impor privilégios via MMU.

- O uso de sections de 1 MB simplifica a TTB e o `map_section(...)`, mas pode desperdiçar memória quando os processos usam pouco espaço, gerando fragmentação interna elevada. Páginas menores seriam mais eficientes, porém exigem tabelas de segundo nível e mais complexidade.
- Permitir que cada processo tenha sua própria seção de código, em vez de uma `.text` única. Isso exigiria ajustar o *linker*/carregamento e o mapeamento (uma entrada de section por código).

8.2. Melhorias

- Permitir que o processo seja mapeado para qualquer seção física livre, independente do TID, e registrar na TCB a tabela/descritor do mapeamento; para que isso seja possível, também seria necessário adicionar um mapa de bits para verificar quais posições da memória física estão disponíveis para alocação. Assim, fica simples reutilizar seções quando um processo termina.
- Implementar o encerramento de processos a partir do stub `_exit`. Para isso, seria necessário chamar a função `exit()` da *libc* dentro do nosso processo, fazendo com que o stub `_exit` seja acionado. A função do stub seria remover a TCB do processo da lista de TCBs e escalonar o processo para executar
- Implementar a criação dinâmica de processos a partir da chamada da `fork()` pela *libc*, que acionaria o stub `_fork`. A função do stub seria:
 - alterar o valor de `r0` na TCB do processo pai por 0;
 - duplicar a TCB do projeto pai alterando o valor de retorno (`r0`) para o PID do processo filho;
 - escolher uma seção livre da memória física e copiar para lá a seção de 1MB relativa a memória do processo pai;
 - adicionar a TCB do processo filho na lista de TCBs;
- Configurar AP bits e Domain/DACR para que páginas do kernel permaneçam somente supervisor, reforçando o isolamento além do “não mapear”.

- Evoluir de sections (1 MB) para páginas de 4 KB com tabelas de segundo nível, reduzindo fragmentação e abrindo espaço para políticas mais finas (a custo de maior complexidade no setup da MMU).

9. Conclusão

O projeto cumpriu o objetivo de adicionar suporte à memória virtual a um escalonador simples em ARM, produzindo um sistema capaz de isolar processos pelo mapeamento por seções de 1 MB. A abordagem manteve a .text de todos os processos fixa no primeiro 1 MB físico (mapeamento flat) e remapeou, a cada switch, a janela virtual 0x100000–0x1FFFFFF para a seção física do processo atual. A integração da MMU em bare-metal exigiu manipular registradores privilegiados (SCTLR, TTBR0/TTBR1, DACR) e cuidar da invalidação de TLB, consolidando o entendimento prático de tradução de endereços.

Os resultados foram validados com tarefas que escrevem/leem variáveis globais. Após correções estruturais, como mover TCBs para a seção .tcb em 0x500000 e isolar a pilha de sistema (stack_sys) fora da janela remapeada, as trocas de contexto estabilizaram. Cada tarefa passou a observar somente os próprios dados, e o fluxo irq/swi → schedule() → task_switch (com mmu_stop, map_section, invalidação de TLB e reprogramação de TTBR/DACR/SCTLR) manteve consistência na volta ao espaço do usuário.

Apesar das simplificações assumidas (código único, seções de 1 MB, proteção baseada em “não mapear”), a base construída é sólida para evoluções futuras e já entrega uma visão clara dos mecanismos de memória virtual em ARM.